University of St.Gallen

Institute of Computer Science

Einführung in Computersysteme, HS2026
Luka Bekavac, Karim Khamaisi, Simon Mayer
{luka.bekavac, karim.khamaisi, simon.mayer}@unisg.ch

Assignment 2: Assembler Forensics (10pt + Bonus)

Deadline: November 22nd, 2026; 23:59 CET

In this assignment, you will go even more bare-metal and program the Raspberry Pi directly using the assembly language, which, as you have seen in the lecture, is just a thin layer of 'syntactic sugar' on top of the binary code that is run by the processor. Note that the ARM A32 instructions that you will see and use in this assignment are different from the x86_64 (i.e., Intel) that we encounter during the lecture.

For this assignment, you need a *Raspberry Pi 400 PC-Kit*, two *330Ω resistors*, two *LEDs*, a *breadboard*, and some *jumper wires*.

Everything you need is in the assignment repository:

https://github.com/Interactions-HSG/2026-HS-ICS-Assignment2

It works exactly like in Assignment 1. Start by reading the `README.md` there – it walks you through creating your own copy, step by step.

You work in your **assigned group of two** and share **one** repository: one of you creates it from the template, the other joins as a collaborator. This is a **new** repository – do not reuse your one from Assignment 1.

By the deadline, you hand in that **GitHub repository**. There is no zip file, and only **one of you** submits – on Canvas you hand in **only the URL of your repository**. It must contain:

- **All tasks:** Your answers in the Markdown[1] file `REPORT.md` – the answers to Tasks 1, 2.1–2.6 and 4, and the explanations of your work in Task 3 and the Bonus Task. All questions below are already written out in that file – you only fill in the blanks. Please also state there your **group number**, the names of **both** team members and their roles, the major pitfalls you encountered, how many minutes/hours you spent on this assignment, and how you used AI.
- **Task 2.7:** Your C code in the file `Task2-3/led.c`
- **Task 3:** Your assembly code in the file `Task2-3/reverse.s`
- **Bonus Task:** Your optimized `flash.elf`, which replaces the given `Bonus/flash.elf`

Three things we cannot grade without, so please do not forget them:

- Your repository must be **private** – use the **Use this template** button on the page above, **do not fork**.
- It must be named **`ics-a2-group<your number>`**, using your group number from Canvas, digits only – for example **`ics-a2-group7`**. We match repositories to groups by that name.
- You must add **your partner**, **`lukabekavac`** and **`Karimkh31`** as **collaborators** on your repository.

[1] https://www.markdownguide.org/

On using AI: for this assignment, you may use AI tools for some Tasks. For example in Task 2 we explicitly ask you to (see there). But, use the AI as a collaborator: work *together* with it to take the binary apart, to work out what the code does, and to debug your own code when it does not behave as expected. Whatever you hand in, you should understand and be able to explain yourself. Note down in `REPORT.md` which tools you used and what for.

(1) Getting Ready (0.5pt)

The assembly code (`Task1/power_led.s`) provided for this task just turns **off** the power LED[2] by writing a value to the given address. To assemble `power_led.s` into an executable format (i.e. ELF[3]), you can use the GNU assembler `arm-none-eabi-as` from the GNU Arm Embedded Toolchain[4] that you have already set up in Assignment 1.

To complete this setup task, use the commands shown in `docs/TOOLCHAIN.md` (or simply run `make power_led`) to deploy `power_led.s` on the Raspberry Pi, and describe in `REPORT.md` what you observe. Use the Arm Architecture Reference Manual Armv8, for Armv8-A architecture profile[5] as a general reference for this assignment.

(2) Assembler Forensics (5.0pt)

Work on this task together with an AI. Reverse engineering a binary is exactly the kind of work where an AI assistant or agent is a useful partner, so for this task we explicitly want you to use one. Work out *together* with it what `kernel.elf` does: let it explain instructions, registers and encodings to you, ask it why the code does what it does, and debug with it when your own code does not behave like the original. But check what it tells you against the manual and against what you see on your Raspberry Pi – AI regularly gets addresses, bit fields and instruction details wrong, and confidently so. In section 0.4 of `REPORT.md`, describe how you worked with it.

In the previous task, you observed that the binary executable is indeed derived from the assembly code – now, let's see how this procedure can be reversed. Analyzing a binary to generate the assembly code is called "disassembling" and is a major type of *reverse engineering* in Computer Science. For this task, we provide the ELF file `Task2-3/kernel.elf` and you need to investigate the program stored in the file.

To support you, the GNU toolchain provides a tool `arm-none-eabi-objdump` that can disassemble ELF binaries. See `docs/TOOLCHAIN.md` for the correct commands.

By disassembling `kernel.elf`, you get the disassembled code similar to the file `kernel.s`:

```
1  kernel.elf:     file format elf32-littlearm
2  kernel.elf
3  architecture: armv8-a, flags 0x00000112:
4  EXEC_P, HAS_SYMS, D_PAGED
5  start address 0x00008000
6
7
8  Disassembly of section .text:
9
10 00008000 <main>:
11     8000:       e30821e4        movw    r2, #33252      ; 0x81e4
12     8004:       e3402001        movt    r2, #1
```

[2] By default, the power LED turns on as you plug in the power source.
[3] https://en.wikipedia.org/wiki/Executable_and_Linkable_Format
[4] https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads
[5] https://developer.arm.com/documentation/ddi0487/fc/ (also provided in `docs/` in the repository)

```
13      ...
```

Listing 1: The beginning of `kernel.s`

- The ELF file is in `elf32-littlearm` format – that is, the addressing is **32-bit** and the bytes are stored in **little-endian**[6].
- The `main` starts at the **`0x8000`** bytes from the beginning of the `kernel.elf` file. **`0x8000`** is the default value for the ARM architecture[7].
- The first instruction stored at the offset `0x8000` is the 32-bit data **`e30821e4`**, which is **`movw r2, #33252`**[8] encoded for the A32 Instruction Set[9].
- The next instruction is then stored at the offset `0x8004` (32 bits=4 bytes after the first).

However, the above raw output format is not compatible with the assembler, and the assembler cannot directly reassemble it to an ELF. For this reason, we provide you the file `Task2-3/kernel_clean.s` that we modified and is ready to be re-assembled.

Now, connect two LEDs in parallel to your Raspberry Pi PIN 38 and 40 (BCM 20 and 21[10]), each with its own 330Ω resistor. Deploy the program stored in `kernel.elf` on your Raspberry Pi (`make kernel` turns it into a `kernel7l.img` for you) and answer the following questions in `REPORT.md`:

1.) (0.5pt) Explain how the 2 LEDs behave. How many patterns do you observe? What do they indicate?

2.) (0.5pt) How many *types* of instructions do you find in the disassembled code (i.e., `kernel.s`)? List all of them with brief explanations about what they are by looking them up in the *AArch32 – Base Instructions*[11]. However, note the following hints about aspects that are not so obvious in the documentation:

- **`BCC`** consists of **`B`** (the Branch instruction[12]) + **`CC`** (the conditional code "Carry clear" also known as "Unsigned lower")[13]. The `bcc` instructions are executed only when the conditional flags[14] satisfy `C == 0`, which are set by the preceding `cmp`[15] instructions.
- The *Constant and immediate values* section of the ARM Cortex-A Series Programmer's Guide[16] should help you understand the set of **`MOVW`** and **`MOVT`** instructions.

3.) (0.5pt) Among the instructions in the code, `b` and `bcc` cause branches (as you saw in the lecture, in x86-64 these are called *jumps*) to target addresses. Unlike in `kernel.s`, the `b` and `bcc` instructions in `kernel_clean.s` take *labels* (e.g., **`.L10`**) to indicate specific addresses to branch to. Compare `kernel.s` and `kernel_clean.s` and for all 9 labels (`.L2` to `.L10`) find out which labels in `kernel_clean.s` correspond to which addresses in `kernel.s`.

[6] https://developer.arm.com/documentation/den0024/a/ARMv8-Registers/Endianness

[7] https://github.com/torvalds/linux/blob/master/arch/arm/Makefile#L145

[8] You can use the online HEX to ARM translater to see it yourself, under the ARM Big Endian: https://armconverter.com/?disasm&code=e30821e4

[9] See F4: *A32 Instruction Set Encoding* in the Arm Architecture Reference Manual[5]

[10] https://pinout.xyz/pinout

[11] https://developer.arm.com/documentation/ddi0597/2021-12/Base-Instructions?lang=en

[12] See F5.1.18: *B* in the Arm Architecture Reference Manual[5]

[13] See F2.3: *Conditional execution* in the Arm Architecture Reference Manual[5]

[14] See E1.2.4: *Process state, PSTATE* in the Arm Architecture Reference Manual[5]

[15] See F5.1.35: *CMP (immediate)* in the Arm Architecture Reference Manual[5]

[16] https://developer.arm.com/documentation/den0013/d/ARM-Thumb-Unified-Assembly-Language-Instructions/Instruction-set-basics/Constant-and-immediate-values?lang=en

4.) (1.0pt) Inspect the instructions in `kernel.s`. Find out the actual execution sequence of instructions when the code runs. Specify the instructions in sequence of execution by giving their addresses. To avoid writing down all instructions, you can "summarize" instructions that are executed in the same sequence as they are specified in `kernel.s`.However, describe how the program branches out from certain locations.

5.) (0.5pt) Which part of `kernel.s` **switches off the two LEDs at the same time**? Identify the part which writes to the corresponding GPIO register, specify it by a range of addresses for the instructions, and explain what values are stored/loaded from where and how.

6.) (1.0pt) The intervals between the lighting patterns of the LEDs are implemented as *loops* and all the loops have the same number of iterations. How many iterations[17] does one loop constitute?

7.) (1.0pt) Write C code that you think the original code for `kernel.elf` looks like. Put it in the file `Task2-3/led.c`, which is already in your repository (still empty). You can build it with `make led` and run it on your Raspberry Pi to see whether it behaves like `kernel.elf`.

(3) Tinkering with Instructions (1.5pt)

You now require a program that cycles through the patterns you figured out in Task 2.1, **but in reverse order**. Your repository already contains **`Task2-3/reverse.s`**, which starts out as an exact copy of `kernel_clean.s`. Modify it so that it implements that behavior. Reassemble the modified assembly code (`make reverse`) and run it on your Raspberry Pi. Your modified `reverse.s` is the hand-in for this task; explain the part(s) you modified in your `REPORT.md`.

[17] Note that `for(int i=0; i<5; i++);` is **1** loop with **5** iterations. Also, see https://en.wikipedia.org/wiki/Iteration

(4) Speak Like a Machine (3.0pt)

Through Tasks 1-3, you have become familiar with assembly language, but what about the language that the machine understands – the binaries? In this task, you will *hand-assemble* A32 instructions to binaries.

1.) (1.5pt) The A32 Instruction Encoding

Taking `kernel7l.img` from **Task 1** as an example, the actual content of the file looks like this:

```
% xxd kernel7l.img
00000000: 0840 9fe5 2c30 84e2 012b a0e3 0020 83e5  .@..,0...+... ..
00000010: 0000 20fe                                ..  .
```

The first instruction is **`ldr r4, [pc, #8]`** which is encoded as **`0xe5_9f_40_08`**. However, you can see that the file starts with **`0x08_40_9f_e5`**. This is because the executable image or the ELF file is created in the `elf32-littlearm` format. But how can we confirm the encoding?

The binary representation of `0xe59f4008` is:
`0b1110_0101_1001_1111_0100_0000_0000_1000` (verify it!).

Following the encoding table in the reference manual[18], the fields for this instruction can be interpreted as follows for the syntax **`LDR{<c>}{<q>}<Rt>,[PC,#{+/-}<imm>]`**.

- *cond*: 1110 (= None)
- *LDR (literal)*: 010PU0W11111
  - *P*: 1 (= Pre-indexed)
  - *U*: 1 (= the offset is added to the base register)
  - *W*: 0 (= The modified base value may not be written back into the base)
- *Rt*: 0100 (= R4)
- *imm12*: 000000001000 (= 8)

Therefore, the `0x0840 0x9fe5` in `kernel.elf` actually do match `ldr r4, [pc, #8]`.

Question: What is the encoding A1 in the `elf32-littlearm` format of the A32 instruction **`ldr r3, [r3]`**? In your `REPORT.md`, also include a brief explanation of each field according to the manual. Note that the *LDR* syntax is different from the above example and is in **`LDR{<c>}{<q>}<Rt>,[<Rn>{,#{+/-}<imm>}]`** which is the A1 Offset variant of *LDR (immediate)*[19] instead of *LDR (literal)*.

2.) (1.5pt) Program Counter (PC) and Offset

When executing an A32 instruction, the Program Counter (PC) holds the address of the current instruction *plus 8 bytes*[20]. For example, in Listing 1 L11–12, the PC starts at **`0x8008`**, executes the instruction stored at the address **`0x8000`**, and then increments to **`0x800C`** (PC values, i.e., the address of the current instruction plus 8, are not `0x8000` then `0x8004`). The *Branch* instructions use the offset between PC and the target address, for example.

Question: What is the encoding A1 in the `elf32-littlearm` format of an A32 instruction **`bcc 0x80e0`** if the instruction is located at the address **`0x810c`**? In your `REPORT.md`, also include a brief explanation of each field according to the manual and how you computed the value for `imm24`.

[18] See F5.1.73: *LDR (literal)* **A1** in the Arm Architecture Reference Manual[5]

[19] See F5.1.72: *LDR (immediate)* in the Arm Architecture Reference Manual[5]

[20] See Section E1.2.3: *The general-purpose registers, and the PC, in AArch32 state* in the Arm Architecture Reference Manual[5]

(5) Bonus: The Smaller The Better

Optimize the provided **`Bonus/flash.elf`** and reduce the size of the unpacked **`kernel7l.img`** (484 bytes) while preserving the original behavior. Hand in your optimized program by **replacing `Bonus/flash.elf`** with it – `docs/TOOLCHAIN.md` shows how to assemble your own version into `flash.elf`. The command `make flash` builds the image and prints its size. Run the original on your Pi before you replace it, so you know which behavior to preserve. The number of points that we award for this task is calculated by the formula:

$$Min((Reduction_of_Program_Size_in_Bytes) * 0.01, 3.0)$$

The *similarity* of your modified binary to `flash.elf` will be judged individually, so please include a detailed explanation about how you approached this question in `REPORT.md`.

Hint: The program in `flash.elf` only interacts with GPIO Pin 42 (the power LED) and no other GPIO pins.